Проектирование архитектуры и конструирование ПО*

Проектирование ОС, Технологии, Инструменты

Лекция 2

Технология АОМК

Компоненты и интерфейсы



Проблематика ПО

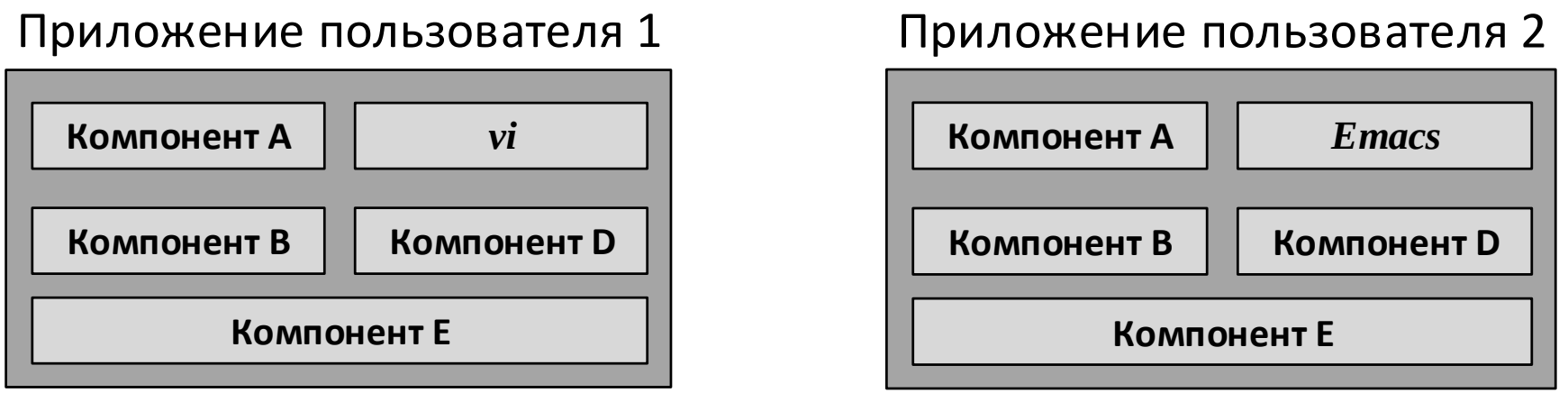
- Устаревание ПО связанное с изменениями:
  - в аппаратной части
  - в операционных системах
  - в модификации прикладных решений
- Решение:
  - разбиение монолитного приложения на компоненты
  - замена компонента на новую, улучшенную версию





Проблематика ПО

- Адаптация ПО
  - построение приложения под нужды пользователей
  - создание специализированных решений на основе готовых продуктов
- Решение:
  - создание приложений из компонентов соответствующим потребностям пользователя



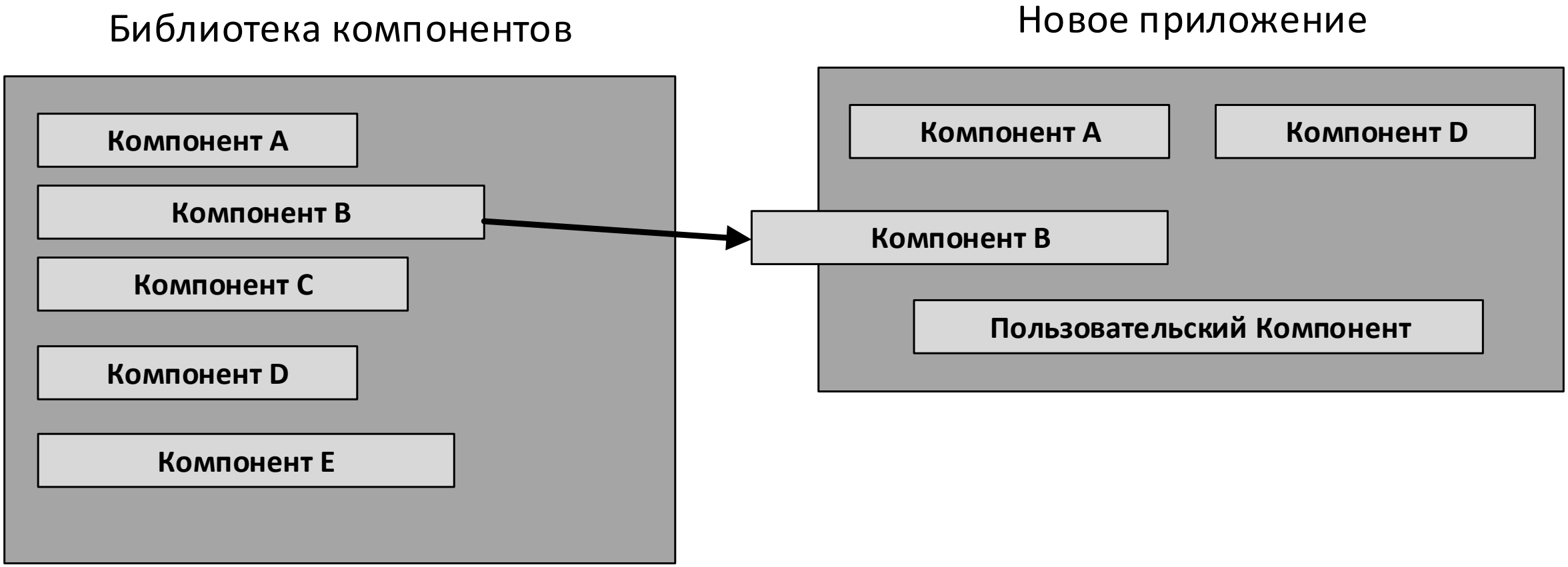


Проблематика ПО

- Отсутствие стандартов компоновки двоичных результатов компиляции в единое целое:
  - нельзя компилировать взаимодействующие объекты разными компиляторами
  - изменение реализации одного из модулей ведет к перекомпоновке всего ПО
- Решение:
  - сборка приложений из стандартных блоков (компонентов)
  - повторная применимость в различных решениях

Проблематика ПО

- Пример использования библиотеки стандартных компонентов (готовых решений) :





ГОСТы и Международные стандарты

- **Понятие «Компонент»**

Сборочное программирование Е.М.Лаврищева, В.Н.Грищенко КИЕВ НАУКОВА ДУМКА 2009 :

Компонент

Программный объект, который реализует некоторую функциональность и является базовым понятием компонентного программирования и компонентно-ориентированной разработки (component-based development – CBD).



ГОСТы и Международные стандарты

- **Понятие «Объект»**

ГОСТ 19781-90: Объект (см. Примечание)

Состоит из данных и операций над данными.

- **Понятие «Компьютерная модель»**

ГОСТ Р 57412-2017: Компьютерная модель

Модель, выполненная в компьютерной (вычислительной) среде и представляющая собой совокупность данных и программного кода, необходимого для работы с данными.

ГОСТы и Международные стандарты

УДК 001.4:681.3.06:006.354 Группа Э00

М Е Ж Г О С У Д А Р С Т В Е Н Н Ы Й С Т А Н Д А Р Т

ОБЕСПЕЧЕНИЕ СИСТЕМ ОБРАБОТКИ ИНФОРМАЦИИ ПРОГРАММНОЕ

Термины и определения

ГОСТ 19781—90

Software of data processing systems.
Terms and definitions

МКС 01.040.35
ОКСТУ 4001

Дата введения 01.01.92

Настоящий стандарт устанавливает термины и определения понятий в области программного обеспечения систем обработки информации.

Термины, установленные настоящим стандартом, обязательны для применения во всех видах документации и литературы, входящих в сферу работ по стандартизации или использующих результаты этих работ.

Настоящий стандарт должен применяться совместно с ГОСТ 15971, ГОСТ 20886, ГОСТ 24402.

1. Стандартизованные термины с определениями приведены в табл. 1.

2. Для каждого понятия установлен один стандартизованный термин. Применение терминов-синонимов стандартизованного термина не допускается. Недопустимые к применению термины-синонимы приведены в табл. 1 в качестве справочных и обозначены пометой "Ндп".

2.1. Для отдельных стандартизованных терминов в табл. 1 приведены в качестве справочных краткие формы, которые разрешается применять в случаях, исключающих возможность их различного толкования.

2.2. Приведенные определения можно, при необходимости, изменять, вводя в них производные признаки, раскрывая значения используемых в них терминов, указывая объекты, входящие в объем определяемого понятия. Изменения не должны нарушать объем и содержание понятий, определенных в данном стандарте.

2.3. В табл. 1 в качестве справочных приведены иноязычные эквиваленты для ряда стандартизованных терминов на английском языке.

3. Алфавитные указатели содержащихся в стандарте терминов на русском и английском языках приведены в табл. 2—3.

4. Термины и определения общих понятий, относящихся к системам обработки информации, управлению обработкой данных и представлению данных, необходимые для понимания текста стандарта, приведены в приложении 1.

5. Стандартизованные термины набраны полужирным шрифтом, их краткая форма – светлым.

Таблица 1

Термин	Определение
Основные понятия	
1. **Программа** Program	Данные, предназначенные для управления конкретными компонентами системы обработки информации в целях реализации определенного алгоритма
2. **Программное обеспечение**	Совокупность программ системы обработки информации и программных документов, необходимых для эксплуатации этих программ
3. **Программирование** Programming	Научная и практическая деятельность по созданию программ

Издание официальное ★ **Перепечатка воспрещена**

© Издательство стандартов, 1990
© СТАНДАРТИНФОРМ, 2010

Примечание — При пользовании настоящим стандартом целесообразно проверить действие ссылочных стандартов в информационной системе общего пользования — на официальном сайте Федерального агентства по техническому регулированию и метрологии в сети Интернет или по ежегодному информационному указателю «Национальные стандарты», который опубликован по состоянию на 1 января текущего года, и по выпускам ежемесячного информационного указателя «Национальные стандарты» за текущий год. Если заменен ссылочный стандарт, на который дана недатированная ссылка, то рекомендуется использовать действующую версию этого стандарта с учетом всех внесенных в данную версию изменений. Если заменен ссылочный стандарт, на который дана датированная ссылка, то рекомендуется использовать версию этого стандарта с указанным выше годом утверждения (принятия). Если после утверждения настоящего стандарта в ссылочный стандарт, на который дана датированная ссылка, внесено изменение, затрагивающее положение, на которое дана ссылка, то это положение рекомендуется применять без учета данного изменения. Если ссылочный стандарт отменен без замены, то положение, в котором дана ссылка на него, рекомендуется применять в части, не затрагивающей эту ссылку.

3 Термины, определения и сокращения

3.1 Термины и определения

В настоящем стандарте применены следующие термины с соответствующими определениями:

3.1.1 **модель:** Сущность, воспроизводящая явление, объект или свойство объекта реального мира[1)].

3.1.2 **объект моделирования:** Явление, объект или свойство объекта реального мира*.

3.1.3 **аспект моделирования:** Отдельное свойство или совокупность свойств объекта моделирования, являющихся предметом исследования с помощью моделирования.

3.1.4 **математическая модель:** Модель, в которой сведения об объекте моделирования представлены в виде математических символов и выражений*.

3.1.5 **информационная модель:** Модель, в которой сведения об объекте моделирования представлены в виде совокупности элементов данных и отношений между ними*.

Примечание — Состав (номенклатура) данных определяется областью интереса разработчика модели и потенциального или реального пользователя.

3.1.6 **моделирование:** Изучение свойств и/или поведения объекта моделирования, выполненное с использованием его моделей*.

3.1.7 **компьютерная модель (электронная модель):** Модель, выполненная в компьютерной (вычислительной) среде и представляющая собой совокупность данных и программного кода, необходимого для работы с данными.

3.1.8 **проверка адекватности компьютерной модели:** Совокупность действий с моделью, результатом которых является подтверждение ее соответствия моделируемому объекту реального мира*.

3.1.9 **контроль результатов компьютерного моделирования:** Совокупность действий, результатом которых является подтверждение соответствия компьютерной реализации модели исходной математической или информационной модели*.

3.1.10 **компьютерная модель изделия:** Компьютерная модель, в которой объектом моделирования является изделие(ия)*.

3.1.11 **компьютерное моделирование изделия:** Моделирование, выполненное с использованием компьютерной модели изделия.

Примечание — Компьютерное моделирование изделия выполняют с целью получения данных, необходимых для принятия решений в процессах разработки, проектирования, производства, сопровождения эксплуатации и других задач в ходе жизненного цикла изделия.

3.2 Сокращения

В настоящем стандарте применены следующие сокращения:

ЖЦ — жизненный цикл;
ИО — информационный объект;
КД — конструкторский документ;
КМ — компьютерная модель;

[1)] Здесь и далее знаком «*» отмечены пункты, к которым даны комментарии в приложении А.



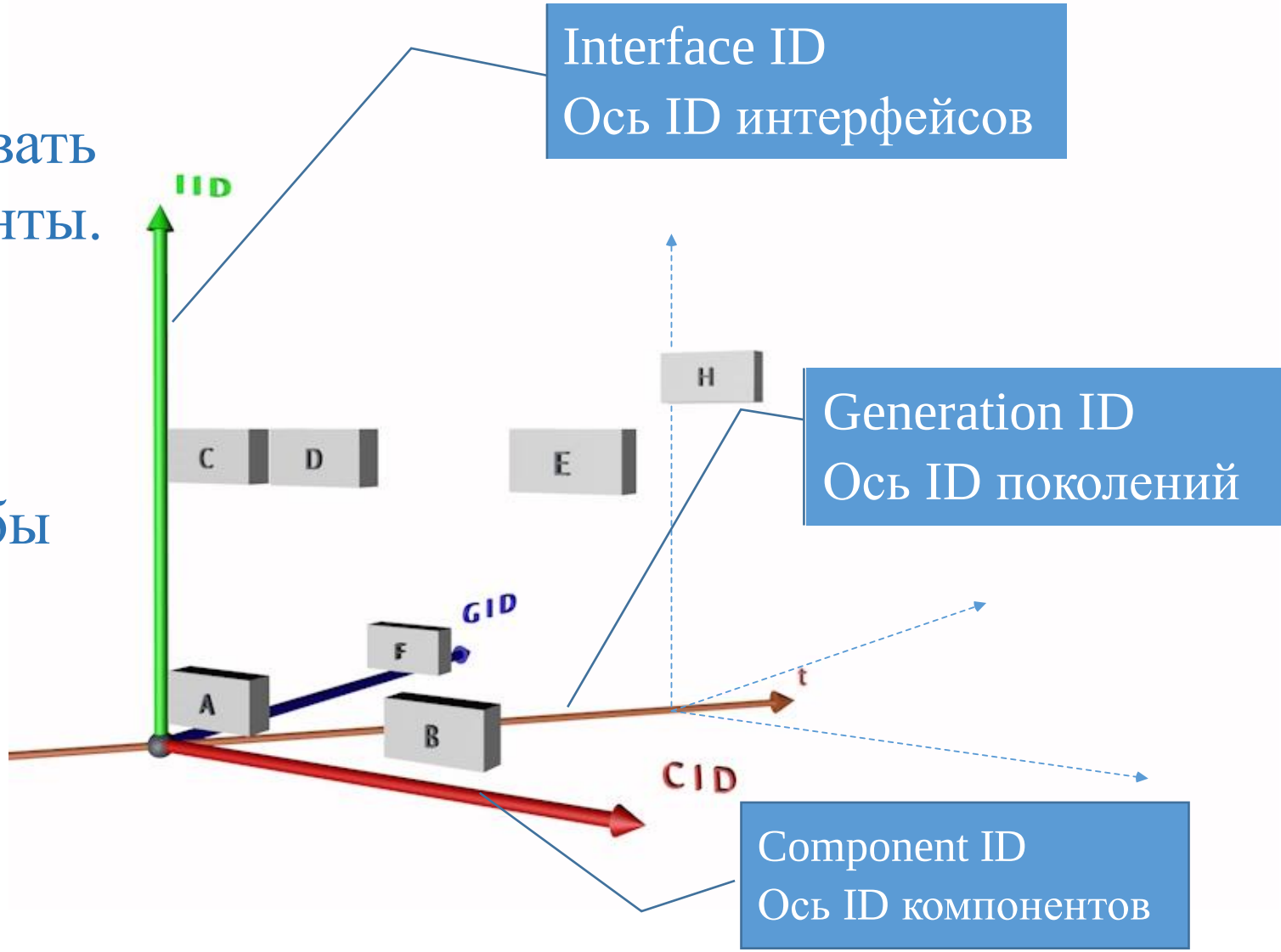
Технология:

Адаптированная Объектная Модель Компонентов (АОМК)

- **АОМК** – это схожая по концепции технология с технологией COM от Microsoft
- **Аналоги:**
  - COM– объектная модель компонентов от Microsoft
  - XPCOM - кроссплатформенная объектная модель компонентов от Mozilla
- **Что общего?**
  - Концепция – общая система взглядов и подходов
  - Наличие методов QueryInterface, AddRef, Release (IUnknown – Microsoft, nsISupport- Mozilla, IEcoUnknown – Отечественные разработки)
- **В чем отличие?**
  - Реализация библиотек
  - Компоненты от Microsoft (COM) несовместимы с компонентами от Mozilla (XPCOM) и не совместимы с отечественными разработками

Технология:
Адаптированная Объектная Модель Компонентов (АОМК)

- АОМК – это спецификация, которая указывает как создавать взаимозаменяемые компоненты.
- АОМК определяет технологический стандарт, которому должны следовать компоненты и клиенты, чтобы гарантировать возможность совместной работы





Почему *Адаптированная* COM технология?

- Компоненты COM состоят из исполняемого кода распространяемого в виде динамически компонуемых библиотек (DLL, SO)

VS

- В адаптированной COM (ACOM), компоненты распространяются как в виде динамических, так и в виде статических библиотек. Что в свою очередь позволяет собирать решения из статических компонентов для микроконтроллеров и систем на кристалле прошивая их (решения) во FLASH-память.



Почему *Адаптированная* COM технология?

- В COM технологии GUID (глобальный уникальны идентификатор) используется в качестве идентификаторов интерфейсов и компонентов и ограничен длиной 128 бита

VS

- В адаптированной COM технологии, используется UGUID (неограниченный глобальный уникальный идентификатор) который может иметь минимальный размер в 16 бит и максимальное значение стремящееся к бесконечности (ограничение технические возможности аппаратной части). Помимо этого UGUID используется не только в качестве идентификатора интерфейсов и компонентов, но и в качестве идентификатора поколений для компонентной ОС.

COM технология – кто первый?



VS





COM технология – кто первый?

Стандарт COM был разработан в 1993 году корпораций Microsoft. Основным понятием, стандарта, является – компонент, который не зависит от языка программирования. Компонент взаимодействует с другими программами через интерфейс.

VS

В 1975 г. начало развиваться сборочное программирование для обеспечения индустрии сборки информационных и программных систем из готовых модулей средствами системы АПРОП (Автоматизированное ПРОизводство Программ) (1975-1992). Базовые идеи были опубликованы в монографии «Связь разноязыковых модулей в ОС ЕС» Е.М. Лаврищева (Финансы и статистика, Москва, 1982). В ней впервые определено понятие интерфейса модулей, стандарт описания модулей в языках (Алгол, Кобол, PL/1, Фортран, Ассемблер) и разработана библиотека из 64 интерфейсных функций преобразования разнотипных данных объединяемых модулей. Данный термин укрепился в программировании и обсуждался на международной конференции «Интерфейс СЭВ» (1987 г.)



ГОСТы и Международные стандарты

- **Понятие «Интерфейс»**

Сборочное программирование Е.М.Лаврищева, В.Н.Грищенко КИЕВ НАУКОВА ДУМКА 2009 :

Интерфейс

Интерфейс – это связь двух отдельных сущностей. Виды интерфейсов: программные, аппаратные, языковые, пользовательские, цифровые и т.п.

Согласно ГОСТу 15971-90 «СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ термины и определения»:

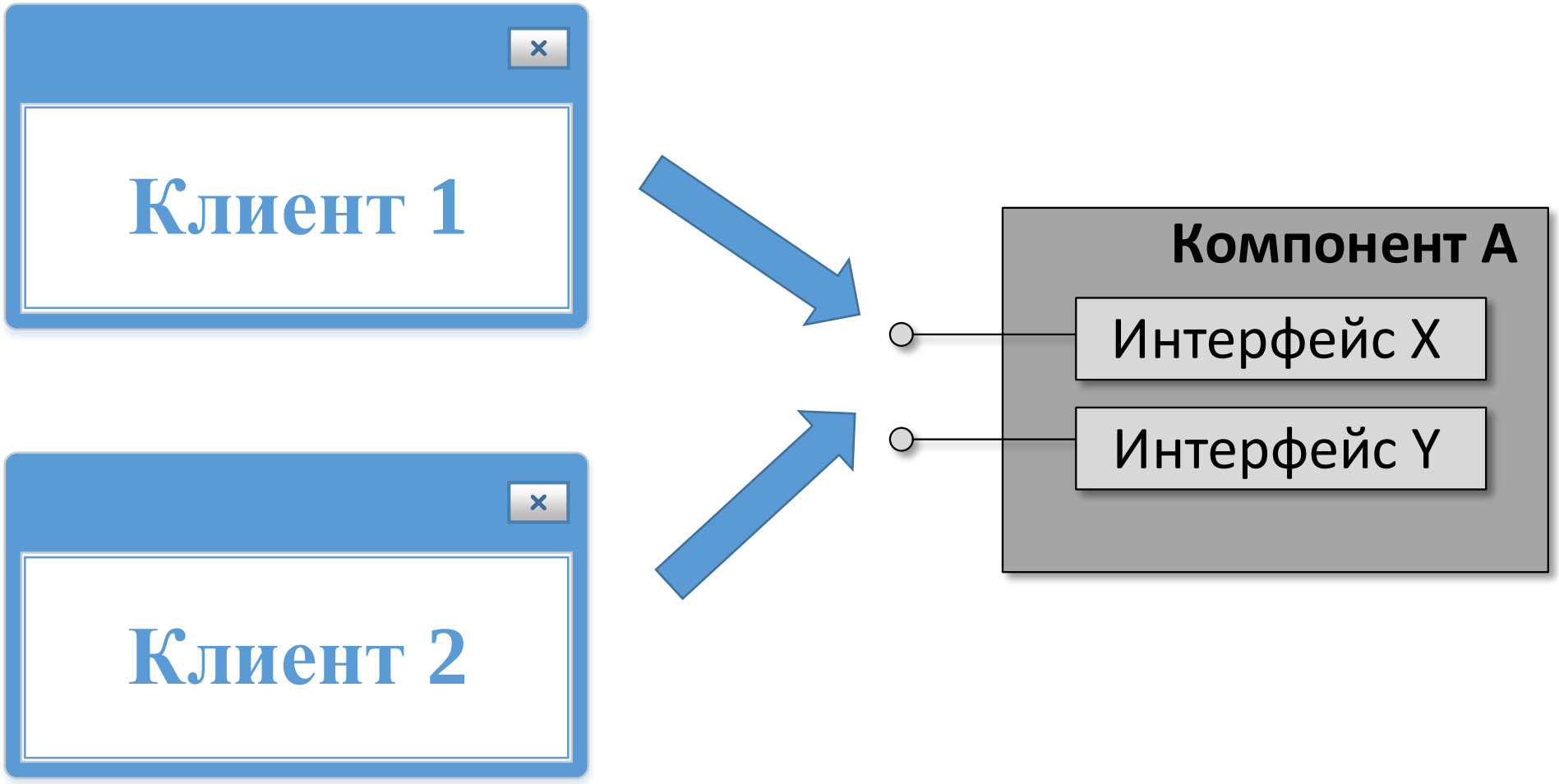
ГОСТ 15971-90: Интерфейс

Совокупность средств и правил, обеспечивающих взаимодействие устройств вычислительной машины или системы обработки информации и (или) программ



Технология:
Адаптированная Объектная Модель Компонентов (АОМК)

Клинт(ы) взаимодействует с компонентом через интерфейс.





Модель компонента

- Под ***моделью компонента*** понимают абстрактное представление, которое отражает основные свойства, характеристики, типичную структуру, а также способность к взаимодействию с другими элементами ***компонентной среды***.
- Модель произвольного компонента в общем случае имеет вид:

  Component = *(CName, CInterface, CFactory, CImplementation, CService)*,

  где ***CName*** – уникальное имя компонента;
  CInterface = $\{CInterface^i\}$ множество интерфейсов, связанных с компонентом;
  CFactory – интерфейс управления экземплярами компонента;
  CImplementation = $\{CImplementation^j\}$ – множество реализаций компонента;
  CService = $\{CService^r\}$ – интерфейс, который определяет множество системных сервисов, необходимых для поддержки функционирования компонента и взаимодействия с компонентной средой.



Модель интерфейса

- Модель интерфейса для каждого *i-интерфейса* компонента имеет вид:

$$\boldsymbol{CInterface^i} = (\boldsymbol{IName^i}, \boldsymbol{IFunction^i}, \boldsymbol{ISpecifucation^i}),$$

где $\boldsymbol{IName^i}$ – имя интерфейса;

$\boldsymbol{IFunction^i}$ – функциональность, реализованная данным интерфейсом (совокупность методов);

$\boldsymbol{ISpecifucation^i}$ – спецификация интерфейса (описания типов, констант, других элементов данных, сигнатур методов и т.д.).



Модель компонентной среды

- Компонентная среда CE (Component Enviroment) определяется как множество элементов

CE = (NameSpace, IRepository, ImpRepository, CService, CServImplementation),

где ***NameSpace*** = {CName^m} – пространство имен, представляющее собой множество имен компонентов в среде;

IRepository = {IRepository^i} – репозитарий интерфейсов, который содержит интерфейсы компонентов среды;

ImpRepository ={ImpRepository^j} – репозитарий реализаций, который содержит реализации для компонентов среды;

CService = {CService^r} – интерфейс, который определяет множество системных сервисов, необходимых для поддержки функционирования компонента;

CServImplementation = {CServImplementation^r} – множество реализаций для системных сервисов.



Внешняя и внутренняя компонентные алгебры

- ***Внешняя алгебра***. Алгебру будем называть внешней, если она определяет операции над компонентами и компонентными средами как над целевыми объектами:
  - инсталляция (развертывание компонента)

    $CE2 = Component \oplus CE1;$
  - объединение компонентных сред

    $CE3 = CE1 \cup CE2;$
  - удаление компонента из компонентной среды

    $CE2 = CE1 \setminus Component.$



Примечание: Для операций внешней компонентной алгебры сформированы и доказаны теоремы, с которыми можно ознакомиться в книге «Сборочное программирование» Е.М.Лаврищева, В.Н.Грищенко, КИЕВ НАУКОВА ДУМКА, 2009



Внешняя и внутренняя компонентные алгебры

- ***Внутренняя алгебра***. Алгебра определяющая операции над отдельными элементами структуры, которые называются операциями эволюции (развития), самой модели компонента:
  - добавление новой реализации для существующего интерфейса;
  - добавление нового интерфейса и новой реализации для него;
  - объединение существующих интерфейсов в один и, если необходимо, то и объединение их реализаций в одну общую реализацию.

Пример:

OldComponent =
(OldCName, OldCInterface, CFactory, OldCImplementation, CService),

NewComponent =
(NewCName, NewCInterface, CFactory, NewCImplementation, CService).



Пример: Операция добавления реализаций

- Операция добавления реализации и интерфейса AddNewImplementation имеет следующую форму записи:

NewComponent =

*AddNewImplementation (OldComponent, NewC*Implementation^{s}*, NewCInterfaceO^{s})*

NewCInterface = OldCInterface $\cup$ *NewCInterfaceO^{s},*

NewCImplementation = OldCImplementation $\cup$ *{NewCImplementations }.*

где *NewCImplementations* – реализация, которая добавляется;

OldCInterfacet – множество входных интерфейсов с *OldCInterface*.

NewCInterfaceO^{s} = {*NewCInterfaceO^{sq}*} – дополнительное множество исходных интерфейсов

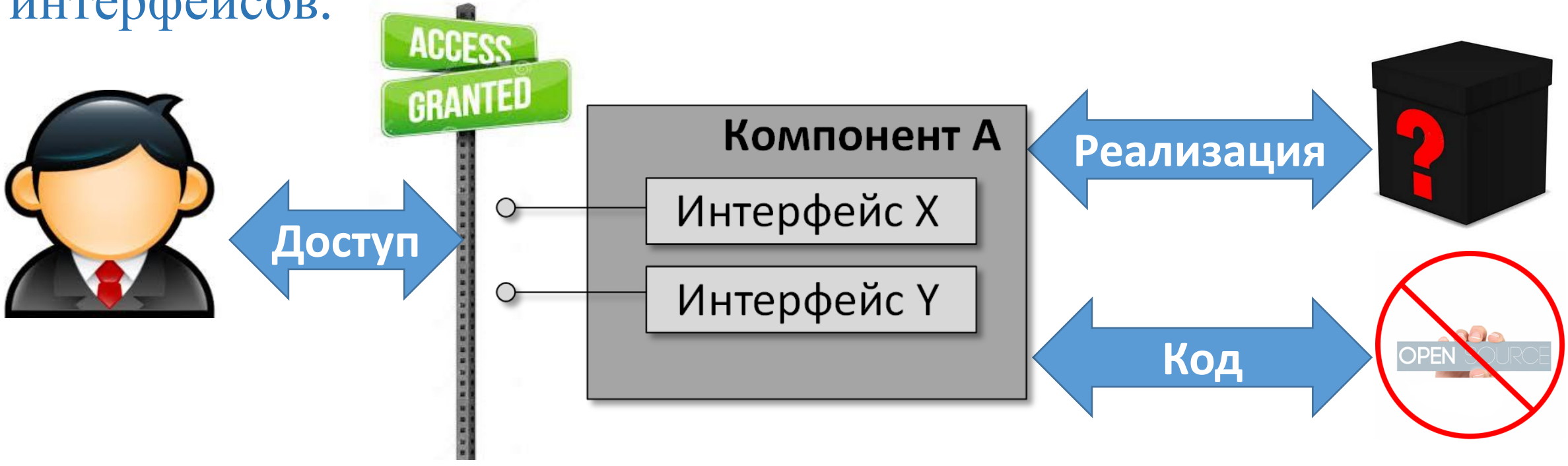
В частном случае, *NewCInterfaceO^{s}* = $\varnothing$, если дополнительная функциональность при реализации, не добавляется.

$\varnothing$ - пустые множества.

Свойства компонентов

1) Компонент - ***инкапсулированный (encapsulated)*** объект.

Инкапсуляция – процесс упрятывания (закрытости) реализации и кода, которые обеспечивают выполнение компонентом своих функций. Инкапсуляция компонента означает, что весь код закрытый, доступ к нему происходит с помощью специальных интерфейсов.

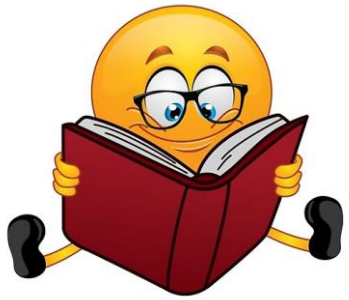


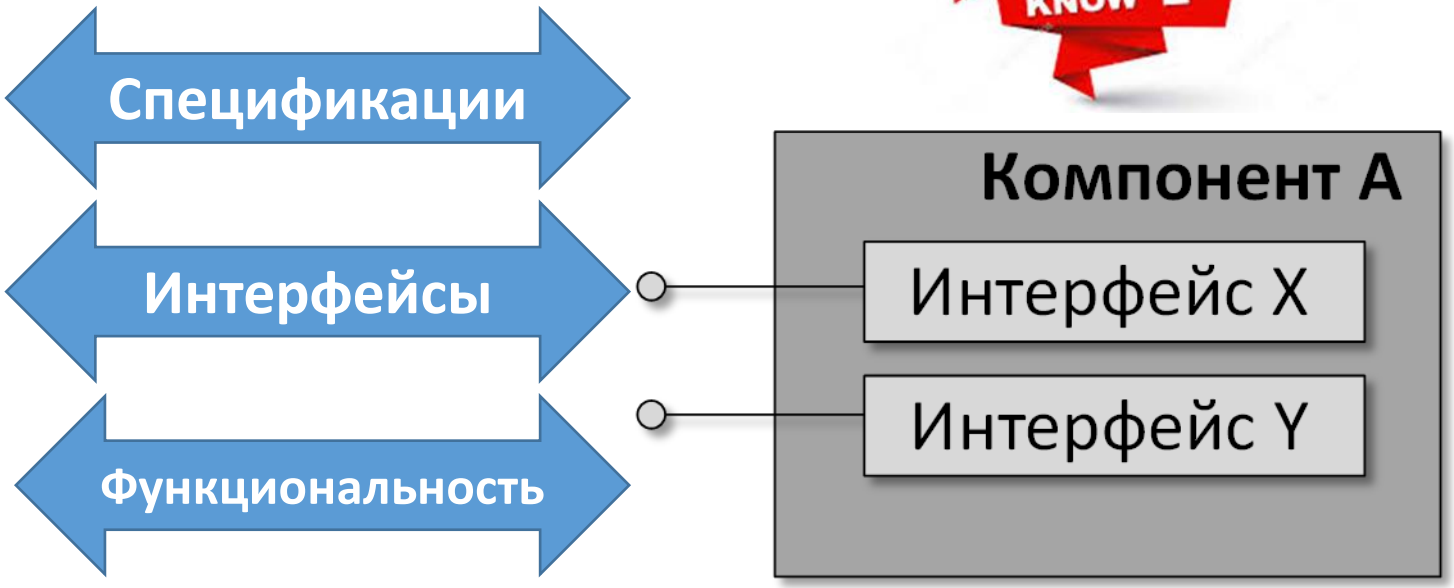
Свойства компонентов

2) Компонент - ***самоопределенный (descriptive)*** объект.

Пользователь компонента должен четко понимать как его назначение, так и особенности функционирования. Этого можно достичь лишь при наличии полной и всесторонней информации о нем.





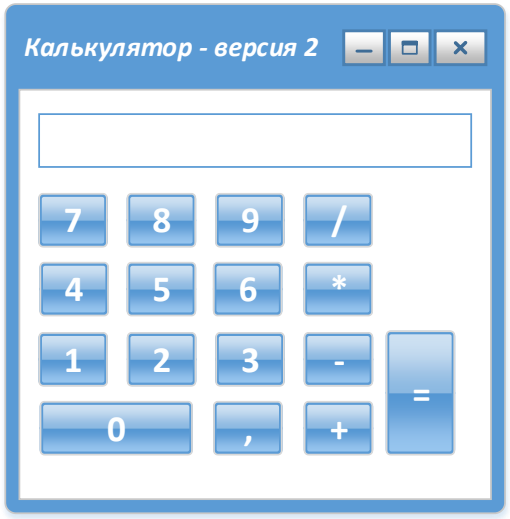


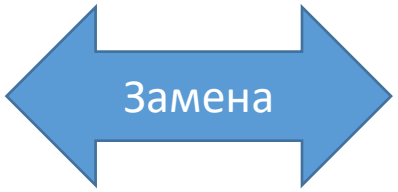


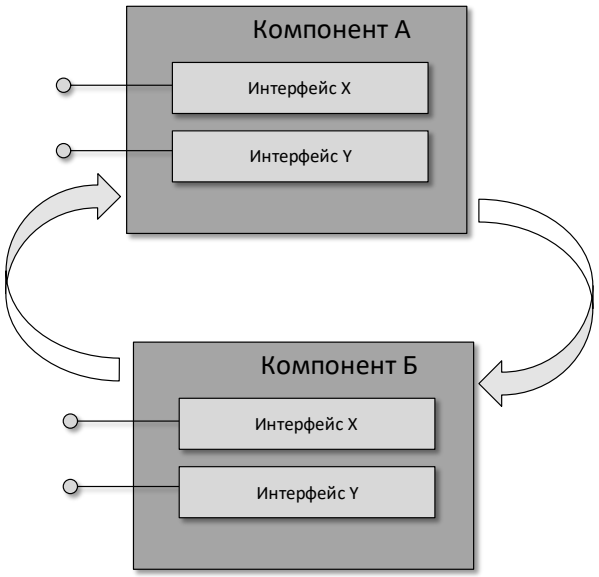
Свойства компонентов

3) Компонент – ***взаимозаменяющий (replaceable)*** объект.

Это свойство является следствием задания инкапсуляции и самоопределения. Первое свойство разрешает пользователю не учитывать внутренние особенности компонента, а второе – определяет его поведение, связи с другими компонентами и требования к внешней среде. В этом случае любой компонент с необходимой функциональностью и соответствующими интерфейсами может быть использован не только при построении программы, но и при сопровождении и развитии (например, новый компонентов более качественно реализует функциональность или требует меньше ресурсов).







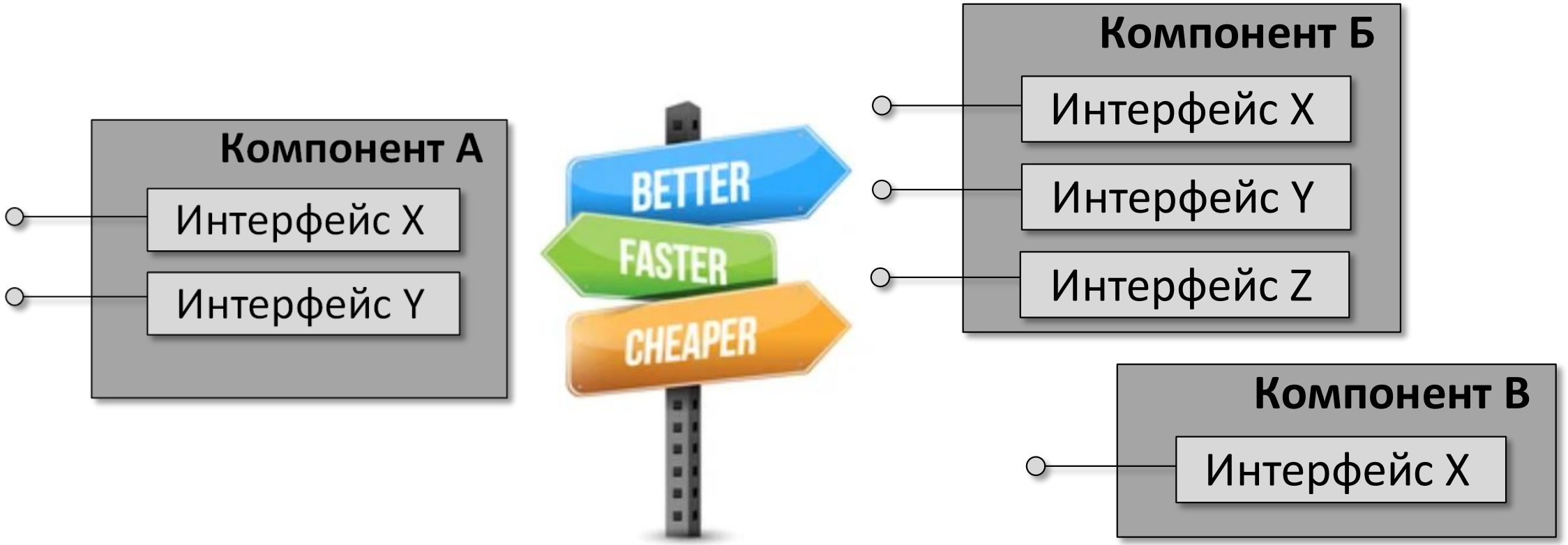




Свойства компонентов

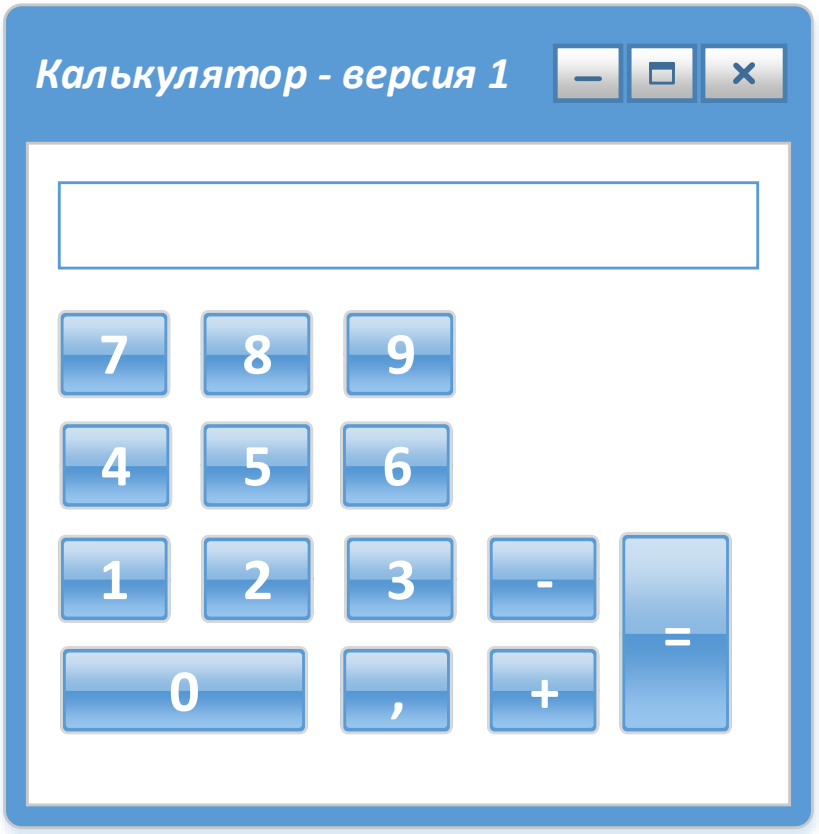
4) Компонент – ***расширяющийся*** (***extensible***) объект.

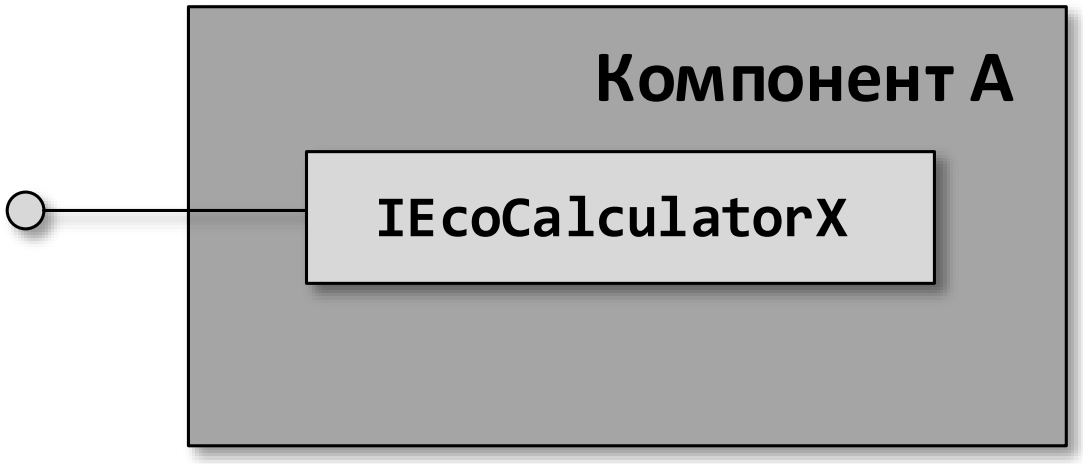
Это свойство обуславливается необходимостью развивать и усовершенствовать компонент в целях улучшения существующих показателей и добавления новых функций.



Пример: модель клиент-сервер

- **Задача:** Необходимо разработать программу Калькулятор с функциями сложения и вычитания, а также с возможностью добавления нового функционала без переписывания кода.







Пример: описание интерфейса на IDL

```
import "IEcoBase1.h"

[
object,
uguid(93221116-2248-4742-AE06-82819447843D),
]

interface IEcoCalculatorX : IEcoUnknown {
    int32_t Addition([in] int16_t a, [in] int16_t b);
    int16_t Subtraction([in] int16_t a, [in] int16_t b);
};
```



Пример: описание интерфейса на Си

```
#include "IEcoBase1.h"

/* IEcoCalculatorX IID = {93221116-2248-4742-AE06-82819447843D} */

typedef struct IEcoCalculatorX* IEcoCalculatorXPtr_t;

typedef struct IEcoCalculatorXVTbl {
    /* IEcoUnknown */
    int16_t (*QueryInterface)(/* in */ IEcoCalculatorXPtr_t me, /* in */ const UGUID* riid,
                              /* out */ void **ppv);
    uint32_t (*AddRef)(/* in */ IEcoCalculatorXPtr_t me);
    uint32_t (*Release)(/* in */ IEcoCalculatorXPtr_t me);
    /* IEcoCalculatorX */
    int32_t (*Addition)(/* in */ IEcoCalculatorXPtr_t me, /* in */ int16_t a, /* in */ int16_t b);
    int16_t (*Subtraction)(/* in */ IEcoCalculatorXPtr_t me, /* in */ int16_t a, /* in */ int16_t b);
} IEcoCalculatorXVTbl, *IEcoCalculatorXVTblPtr;

interface IEcoCalculatorX {
    struct IEcoCalculatorXVTbl *pVTbl;
} IEcoCalculatorX;
```



Пример: описание интерфейса на C++

```
#include "IEcoBase1.h"

/* IEcoCalculatorX IID = {93221116-2248-4742-AE06-82819447843D} */
#ifndef __IID_IEcoCalculatorX
static const UGUID IID_IEcoCalculatorX = {0x01, 0x10, 0x93, 0x22, 0x11, 0x16,
        0x22, 0x48, 0x47, 0x42, 0xAE, 0x06, 0x82, 0x81, 0x94, 0x47, 0x84, 0x3D};
#endif /* __IID_IEcoCalculatorX */

interface IEcoCalculatorX : public IEcoUnknown {

    /* IEcoCalculatorX */
    virtual int32_t Addition(/* in */ int16_t a, /* in */ int16_t b) = 0;
    virtual int16_t Subtraction(/* in */ int16_t a, /* in */ int16_t b) = 0;
};
```



Пример: описание интерфейса на Python

```
import ecosystem1.server

class IEcoCalculatorX

    _public_methods_ = [ 'Addition' 'Subtraction' ]
    _reg_iid_ = "{93221116-2248-4742-AE06-82819447843D}"

    def Addition (self, a, b)
      . . .

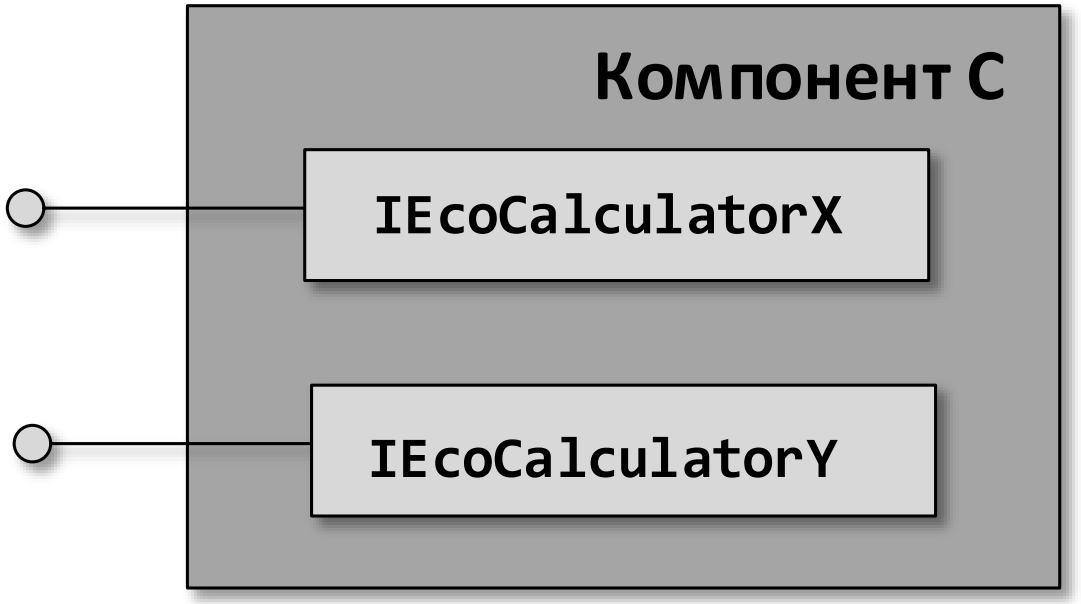
    def Subtraction (self, a, b)
      . . .
```



Пример: Новая версия

- **Задача:** Необходимо доработать программу Калькулятор добавив функции умножения и деления, при этом оставив обратную совместимость с предыдущей версией.







Пример: описание интерфейса на Си

```
#include "IEcoBase1.h"

/* IEcoCalculatorY IID = {BD6414C2-9096-423E-A90C-04D77AFD1CAD} */

typedef struct IEcoCalculatorY* IEcoCalculatorYPtr_t;

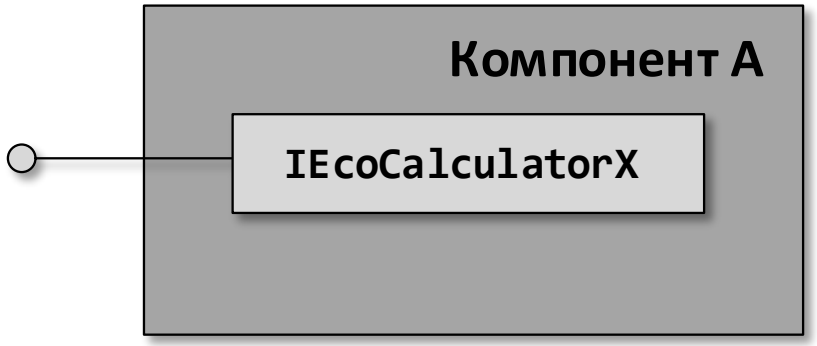
typedef struct IEcoCalculatorYVTbl {
    /* IEcoUnknown */
    int16_t (*QueryInterface)(/* in */ IEcoCalculatorYPtr_t me, /* in */ const UGUID* riid,
                              /* out */ void **ppv);
    uint32_t (*AddRef)(/* in */ IEcoCalculatorYPtr_t me);
    uint32_t (*Release)(/* in */ IEcoCalculatorYPtr_t me);
    /* IEcoCalculatorY */
    int32_t (*Multiplication)(/* in */ IEcoCalculatorYPtr_t me, /* in */int16_t a, /* in */int16_t b);
    int16_t (*Division)(/* in */ IEcoCalculatorYPtr_t me, /* in */ int16_t a, /* in */ int16_t b);
} IEcoCalculatorYVTbl, *IEcoCalculatorYVTblPtr;

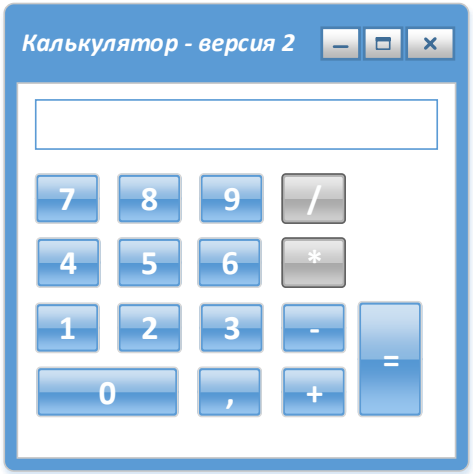
interface IEcoCalculatorY {
    struct IEcoCalculatorYVTbl *pVTbl;
} IEcoCalculatorY;
```



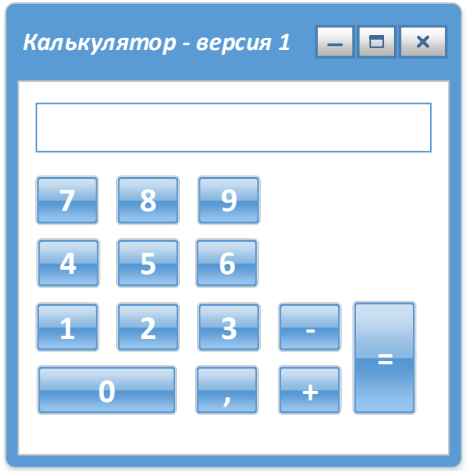
Пример: варианты использования

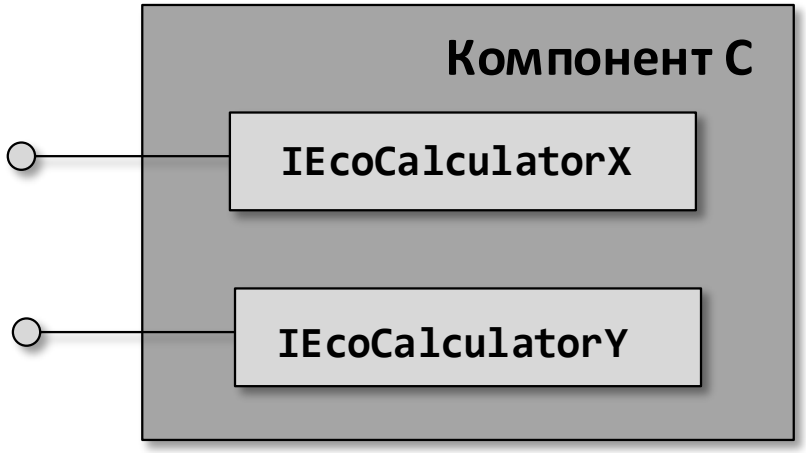


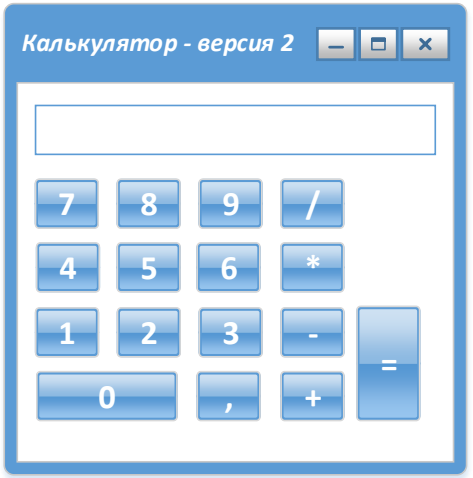




Калькулятор версии 2 использует интерфейс ***IEcoCalculatorX***, доступны только две функции сложение и вычитание.







Калькулятор версии 1 использует интерфейс ***IEcoCalculatorX*** и ничего не знает о интерфейсе ***IEcoCalculatorY***.



Интерфейсы компонентов

Главное ПРАВИЛО (ЗАКОН) технологии АОМК:

– интерфейсы 

НЕЛЬЗЯ изменять,

причем 

НЕЛЬЗЯ изменять никогда.

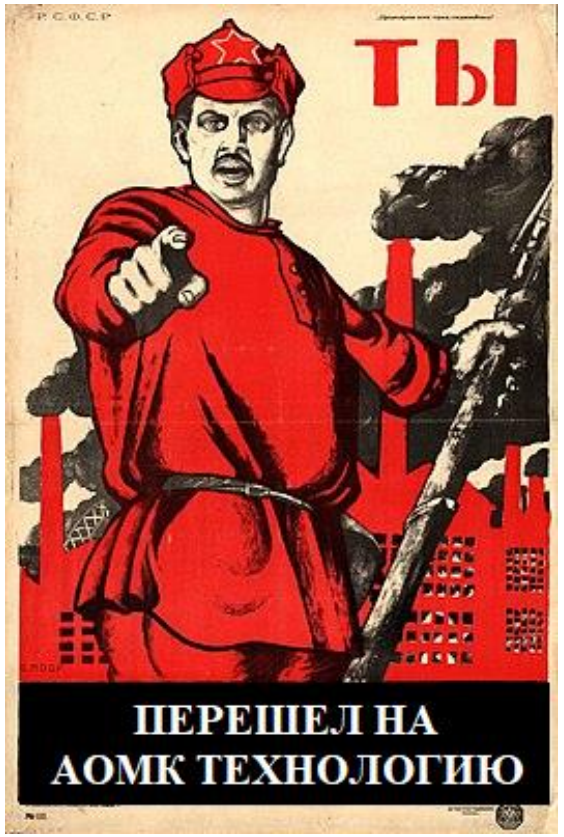
- После публикации интерфейса, он обязан быть всегда тем же самым.
- При модификации компонента следует не изменять существующий, а создать и добавить новый.
- Множественные интерфейсы позволяют компоненту поддерживать новые интерфейсы в дополнении к старым. Обеспечивая прочный фундамент, на основе которого клиенты и компоненты могут воспринимать новые версии партнеров.



Технология АОМК

АГИТАЦИЯ и ПРОПАГАНДА

Надоело переписывать код, используя сторонние библиотеки с меняющимся API!? Не уверен в завтрашнем дне, встречая объявленные функции как DEPRECATED!? Вступай в ряды добровольцев АОМК технологии!!!





Что за интерфейсом?

- Определение интерфейса IX (слева) на IDL (Interface Definition Language) :

```
interface IX {

    void Fx1();
    void Fx2();
    void Fx3();
    void Fx4();

};
```

Генерация на Си

```
typedef struct IXVTbl {

    void (*Fx1)(/* in */ struct IX* me);
    void (*Fx2)(/* in */ struct IX* me);
    void (*Fx3)(/* in */ struct IX* me);
    void (*Fx4)(/* in */ struct IX* me);

} IXVTbl, *IXVTblPtr;

interface IX {
    struct IXVTbl *pVTbl;
} IX;
```

Что за интерфейсом?

- Абстрактное определение интерфейса задает формат данных в памяти. Память для структуры не выделяется до тех пор, пока абстрактное представление интерфейса не будет реализовано в производном объекте.

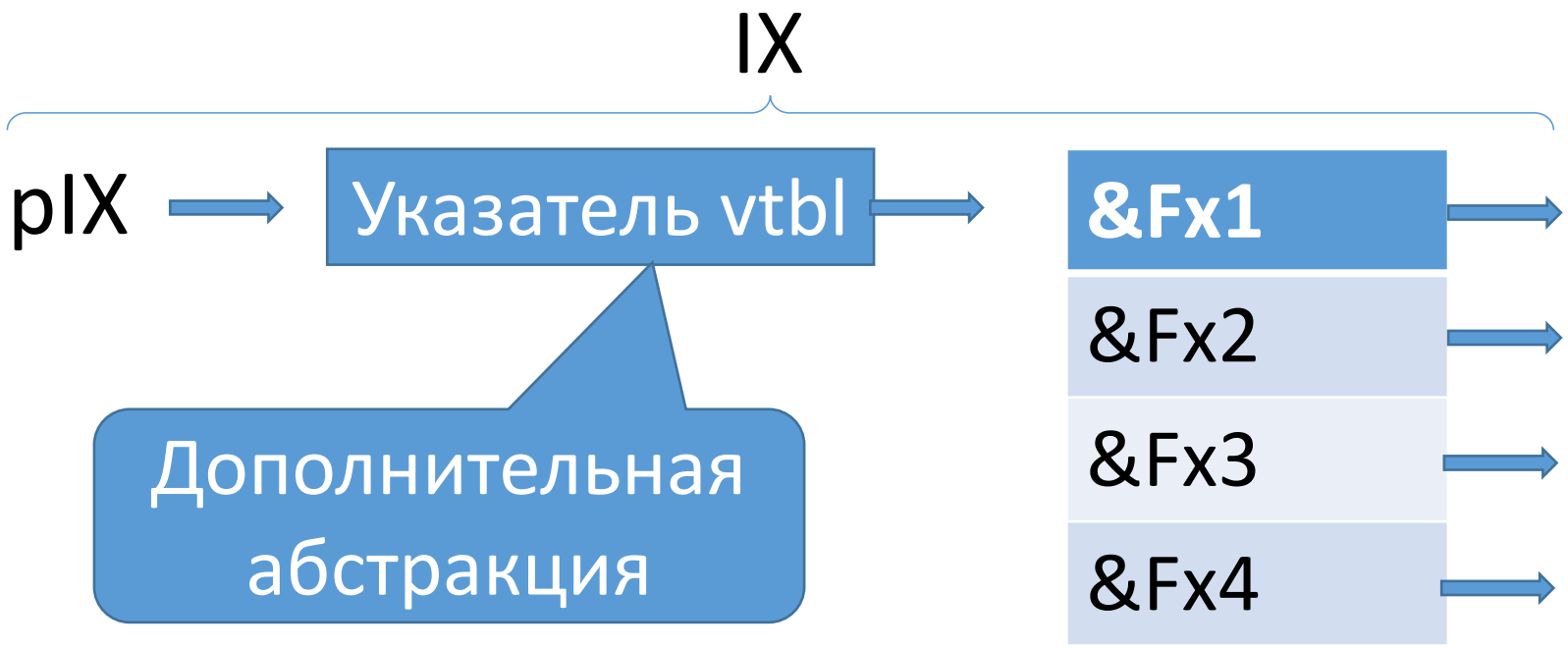


Таблица виртуальных функций



Указатели и экземпляры (реализация)

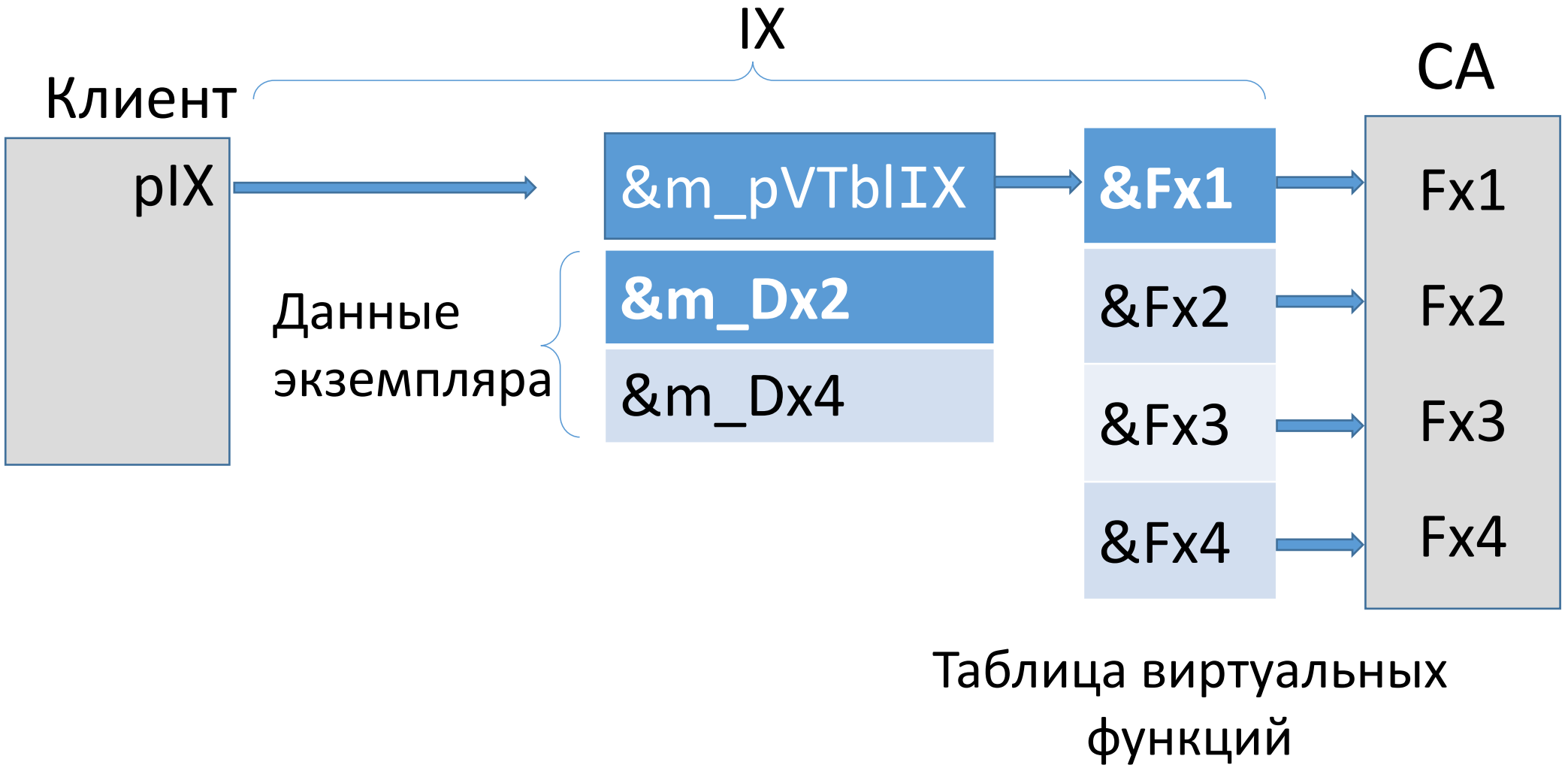
- Реализация абстрактного представления интерфейса :

```
typedef struct CA {
    /* Виртуальная таблица функций интерфейса IX */
    IXVTbl* m_pVTblIX;
    /* Данные экземпляра */
    int m_Dx2;
    int m_Dx4;
} CA, *CAPtr;
```



Указатели и экземпляры (реализация)

- Данные объекта генерируемые компилятором





Преимущества компонентов в программировании:

- снижение затрат на разработку программных систем (ПС);
- сокращение времени разработки отдельных компонентов;
- улучшение показателей качества и надежности программ из компонентов;
- упрощение процедур и снижение затрат на поддержку и сопровождение программных продуктов;
- создание определенной инфраструктуры в разработке ПС, основанной на унификации и стандартизации компонентов в целях практического применения;
- создание рыночных компонентов за счет их стандартизации и ориентации на повторное использование.



Проблематика интерфейсов

- Конфликт имен
  - Компонент может поддерживать несколько интерфейсов, легко возникает конфликт имен функций этих интерфейсов

  В действительности эта проблема относится к используемым языкам программирования, нежели к АОМК технологии. Интерфейсы АОМК технологии отвечают двоичному стандарту. Клиент подключается к интерфейсу не по имени самого интерфейса или его функций, а по ее адресу в блоке памяти.
- Решение:
  - Отказ от множественного наследования. Компонент может содержать указатели на другие объекты реализующие отдельные интерфейсы;
  - Придерживаться стилистического соглашения: - начинайте имена интерфейсов и функций с названием Вашей фирмы или программного продукта. Например, вместо ICalculator1 для продукта Xyz, можно было бы использовать IXyzCalculator1.

  Другой пример использование в компонентной среде CE, такой как Эко (-система):

  IEcoString1, где

  I – обозначение интерфейса, сокращение от Interface

  Eco – уникальный префикс, название компонентной среды (экосистема), сокращение от Ecosystem

  String – имя интерфейса, работа со строкой

  1- версия интерфейса или уникальный постфикс (XXXXX, Ver1Mod5, F5G6…)

Спасибо за внимание!